In [2]:
```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

```
2026-04-08 14:12:43.138519: E external/local_xla/xla/stream_executor/cuda/cud
a_fft.cc:467] Unable to register cuFFT factory: Attempting to register factory
for plugin cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1775657563.524614      55 cuda_dnn.cc:8579] Unable to register cuDN
N factory: Attempting to register factory for plugin cuDNN when one has already
been registered
E0000 00:00:1775657563.632789      55 cuda_blas.cc:1407] Unable to register cuB
LAS factory: Attempting to register factory for plugin cuBLAS when one has alre
ady been registered
W0000 00:00:1775657564.562856      55 computation_placer.cc:177] computation pl
acer already registered. Please check linkage and avoid linking the same target
more than once.
W0000 00:00:1775657564.562915      55 computation_placer.cc:177] computation pl
acer already registered. Please check linkage and avoid linking the same target
more than once.
W0000 00:00:1775657564.562919      55 computation_placer.cc:177] computation pl
acer already registered. Please check linkage and avoid linking the same target
more than once.
W0000 00:00:1775657564.562922      55 computation_placer.cc:177] computation pl
acer already registered. Please check linkage and avoid linking the same target
more than once.
```

In [3]:
```
max_features = 10000

(x_train, y_train), (x_test, y_test) = keras.datasets.imdb.load_data(num_words
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datase
ts/imdb.npz
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

In [4]:
```
maxlen = 200

x_train = keras.preprocessing.sequence.pad_sequences(x_train, maxlen=maxlen)
x_test = keras.preprocessing.sequence.pad_sequences(x_test, maxlen=maxlen)
```

In [5]:
```
model = keras.Sequential([
    layers.Embedding(input_dim=max_features, output_dim=128, input_length=maxl

    layers.LSTM(64, return_sequences=False),

    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5),

    layers.Dense(1, activation='sigmoid')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97:
UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
2026-04-08 14:13:25.845785: E external/local_xla/xla/stream_executor/cuda/cud
a_platform.cc:51] failed call to cuInit: INTERNAL: CUDA error: Failed call to c
uInit: UNKNOWN ERROR (303)
```

In [6]:
```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

In [7]:
```
history = model.fit(
    x_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)
```

```
Epoch 1/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 55s 165ms/step - accuracy: 0.7094 - loss: 0.5418 -
val_accuracy: 0.8600 - val_loss: 0.3360
Epoch 2/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 49s 156ms/step - accuracy: 0.9031 - loss: 0.2611 -
val_accuracy: 0.8410 - val_loss: 0.3518
Epoch 3/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 49s 158ms/step - accuracy: 0.9386 - loss: 0.1796 -
val_accuracy: 0.8580 - val_loss: 0.4659
Epoch 4/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 50s 159ms/step - accuracy: 0.9537 - loss: 0.1352 -
val_accuracy: 0.8642 - val_loss: 0.4185
Epoch 5/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 48s 152ms/step - accuracy: 0.9698 - loss: 0.0880 -
val_accuracy: 0.8486 - val_loss: 0.4535
```

In [8]:
```
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test Accuracy:", test_acc)
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 28s 35ms/step - accuracy: 0.8461 - loss: 0.4684
Test Accuracy: 0.8470399975776672
```

In [9]:
```
embedding_dims = [64, 128]
lstm_units_list = [32, 64, 128]
dropouts = [0.3, 0.5]

results = []
```

In [10]:
```
for emb_dim in embedding_dims:
    for lstm_units in lstm_units_list:
        for dropout_rate in dropouts:

            print("\nTesting config:")
            print(f"Embedding: {emb_dim}, LSTM: {lstm_units}, Dropout: {dropou
```

```
            model = keras.Sequential([
                layers.Embedding(input_dim=max_features, output_dim=emb_dim, i

                layers.LSTM(lstm_units),

                layers.Dense(64, activation='relu'),
                layers.Dropout(dropout_rate),

                layers.Dense(1, activation='sigmoid')
            ])

            model.compile(
                optimizer='adam',
                loss='binary_crossentropy',
                metrics=['accuracy']
            )

            history = model.fit(
                x_train, y_train,
                epochs=3,
                batch_size=64,
                validation_split=0.2,
                verbose=1
            )

            test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)

            print(f"Test Accuracy: {test_acc:.4f}")


            results.append({
                "embedding_dim": emb_dim,
                "lstm_units": lstm_units,
                "dropout": dropout_rate,
                "test_accuracy": test_acc
            })
```

```
Testing config:
Embedding: 64, LSTM: 32, Dropout: 0.3
Epoch 1/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 30s 86ms/step - accuracy: 0.6706 - loss: 0.5687 -
val_accuracy: 0.8192 - val_loss: 0.4067
Epoch 2/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 26s 84ms/step - accuracy: 0.9030 - loss: 0.2565 -
val_accuracy: 0.8758 - val_loss: 0.3022
Epoch 3/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 27s 85ms/step - accuracy: 0.9417 - loss: 0.1609 -
val_accuracy: 0.8664 - val_loss: 0.3235
Test Accuracy: 0.8644

Testing config:
Embedding: 64, LSTM: 32, Dropout: 0.5
Epoch 1/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 30s 87ms/step - accuracy: 0.6850 - loss: 0.5636 -
val_accuracy: 0.8538 - val_loss: 0.3370
Epoch 2/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 27s 87ms/step - accuracy: 0.9128 - loss: 0.2348 -
val_accuracy: 0.8824 - val_loss: 0.2982
Epoch 3/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 27s 85ms/step - accuracy: 0.9425 - loss: 0.1633 -
val_accuracy: 0.8720 - val_loss: 0.3590
Test Accuracy: 0.8634

Testing config:
Embedding: 64, LSTM: 64, Dropout: 0.3
Epoch 1/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 42s 127ms/step - accuracy: 0.7025 - loss: 0.5368 -
val_accuracy: 0.8636 - val_loss: 0.3241
Epoch 2/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 39s 123ms/step - accuracy: 0.9071 - loss: 0.2361 -
val_accuracy: 0.8728 - val_loss: 0.3296
Epoch 3/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 38s 122ms/step - accuracy: 0.9361 - loss: 0.1734 -
val_accuracy: 0.8750 - val_loss: 0.3482
Test Accuracy: 0.8642

Testing config:
Embedding: 64, LSTM: 64, Dropout: 0.5
Epoch 1/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 40s 121ms/step - accuracy: 0.6778 - loss: 0.5577 -
val_accuracy: 0.8666 - val_loss: 0.3105
Epoch 2/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 38s 120ms/step - accuracy: 0.9123 - loss: 0.2375 -
val_accuracy: 0.8758 - val_loss: 0.3046
Epoch 3/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 38s 120ms/step - accuracy: 0.9396 - loss: 0.1671 -
val_accuracy: 0.8592 - val_loss: 0.3516
Test Accuracy: 0.8513

Testing config:
Embedding: 64, LSTM: 128, Dropout: 0.3
```

```
Epoch 1/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 90s 280ms/step - accuracy: 0.6642 - loss: 0.6003 -
val_accuracy: 0.8534 - val_loss: 0.3395
Epoch 2/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 90s 286ms/step - accuracy: 0.9028 - loss: 0.2550 -
val_accuracy: 0.8568 - val_loss: 0.3393
Epoch 3/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 97s 309ms/step - accuracy: 0.9320 - loss: 0.1854 -
val_accuracy: 0.8616 - val_loss: 0.3395
Test Accuracy: 0.8590

Testing config:
Embedding: 64, LSTM: 128, Dropout: 0.5
Epoch 1/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 96s 295ms/step - accuracy: 0.6347 - loss: 0.6112 -
val_accuracy: 0.8516 - val_loss: 0.3506
Epoch 2/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 91s 291ms/step - accuracy: 0.8931 - loss: 0.2731 -
val_accuracy: 0.8582 - val_loss: 0.3410
Epoch 3/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 87s 279ms/step - accuracy: 0.9300 - loss: 0.1911 -
val_accuracy: 0.8724 - val_loss: 0.4138
Test Accuracy: 0.8577

Testing config:
Embedding: 128, LSTM: 32, Dropout: 0.3
Epoch 1/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 33s 96ms/step - accuracy: 0.7162 - loss: 0.5375 -
val_accuracy: 0.8690 - val_loss: 0.3153
Epoch 2/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 29s 92ms/step - accuracy: 0.9112 - loss: 0.2355 -
val_accuracy: 0.8792 - val_loss: 0.3084
Epoch 3/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 29s 94ms/step - accuracy: 0.9464 - loss: 0.1489 -
val_accuracy: 0.8710 - val_loss: 0.3414
Test Accuracy: 0.8658

Testing config:
Embedding: 128, LSTM: 32, Dropout: 0.5
Epoch 1/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 32s 93ms/step - accuracy: 0.6491 - loss: 0.5984 -
val_accuracy: 0.8466 - val_loss: 0.3705
Epoch 2/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 30s 95ms/step - accuracy: 0.9024 - loss: 0.2564 -
val_accuracy: 0.8780 - val_loss: 0.3005
Epoch 3/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 30s 96ms/step - accuracy: 0.9392 - loss: 0.1694 -
val_accuracy: 0.8734 - val_loss: 0.3217
Test Accuracy: 0.8691

Testing config:
Embedding: 128, LSTM: 64, Dropout: 0.3
Epoch 1/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 53s 161ms/step - accuracy: 0.7105 - loss: 0.5274 -
```

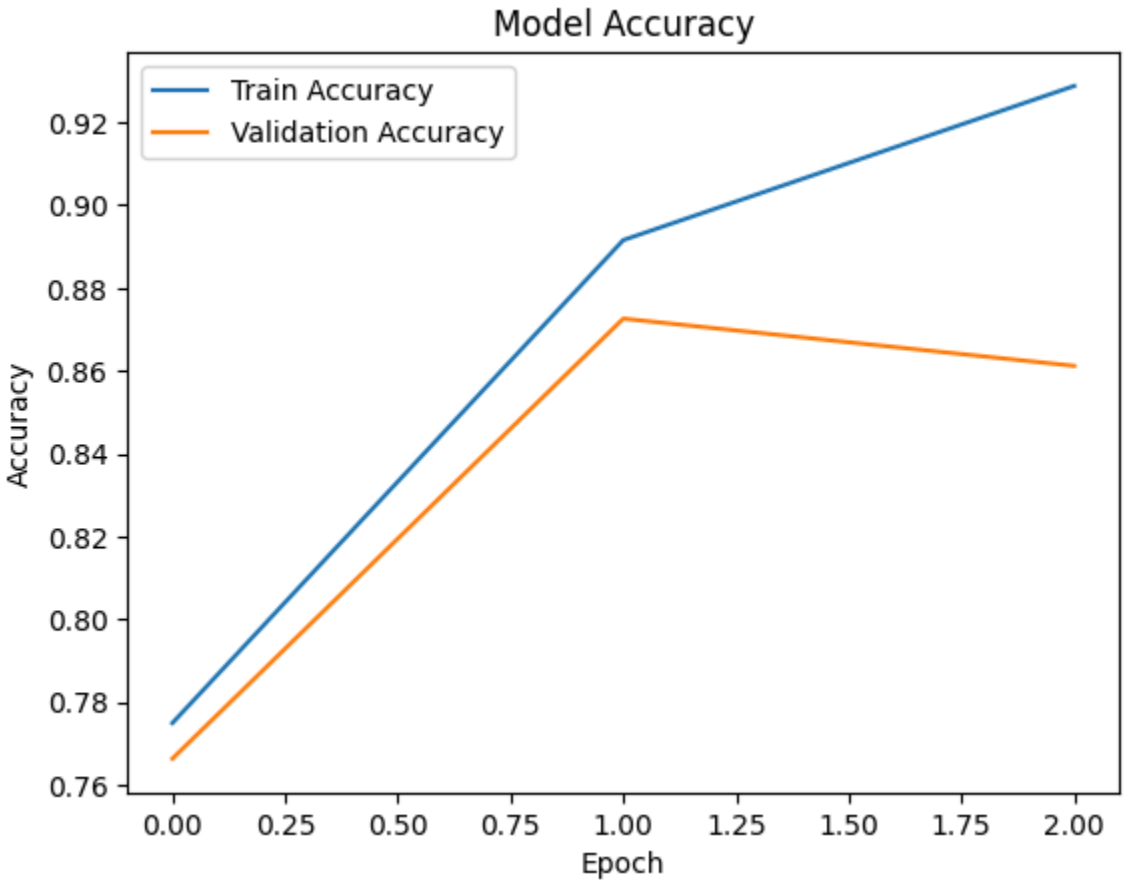
```
val_accuracy: 0.8558 - val_loss: 0.3367
Epoch 2/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 51s 161ms/step - accuracy: 0.9059 - loss: 0.2365 -
val_accuracy: 0.8730 - val_loss: 0.3064
Epoch 3/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 48s 154ms/step - accuracy: 0.9432 - loss: 0.1599 -
val_accuracy: 0.8634 - val_loss: 0.3794
Test Accuracy: 0.8617

Testing config:
Embedding: 128, LSTM: 64, Dropout: 0.5
Epoch 1/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 49s 158ms/step - accuracy: 0.9357 - loss: 0.1801 -
val_accuracy: 0.8722 - val_loss: 0.3438
Test Accuracy: 0.8659

Testing config:
Embedding: 128, LSTM: 128, Dropout: 0.3
Epoch 1/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 101s 314ms/step - accuracy: 0.6645 - loss: 0.5837
- val_accuracy: 0.8324 - val_loss: 0.3981
Epoch 2/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 96s 308ms/step - accuracy: 0.8965 - loss: 0.2619 -
val_accuracy: 0.8726 - val_loss: 0.3114
Epoch 3/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 95s 305ms/step - accuracy: 0.9301 - loss: 0.1921 -
val_accuracy: 0.8524 - val_loss: 0.3590
Test Accuracy: 0.8528

Testing config:
Embedding: 128, LSTM: 128, Dropout: 0.5
Epoch 1/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 100s 312ms/step - accuracy: 0.6853 - loss: 0.5647
- val_accuracy: 0.7664 - val_loss: 0.4714
Epoch 2/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 100s 318ms/step - accuracy: 0.8851 - loss: 0.3011
- val_accuracy: 0.8726 - val_loss: 0.3074
Epoch 3/3
313/313 ━━━━━━━━━━━━━━━━━━━━ 102s 327ms/step - accuracy: 0.9337 - loss: 0.1843
- val_accuracy: 0.8612 - val_loss: 0.3542
Test Accuracy: 0.8563
```

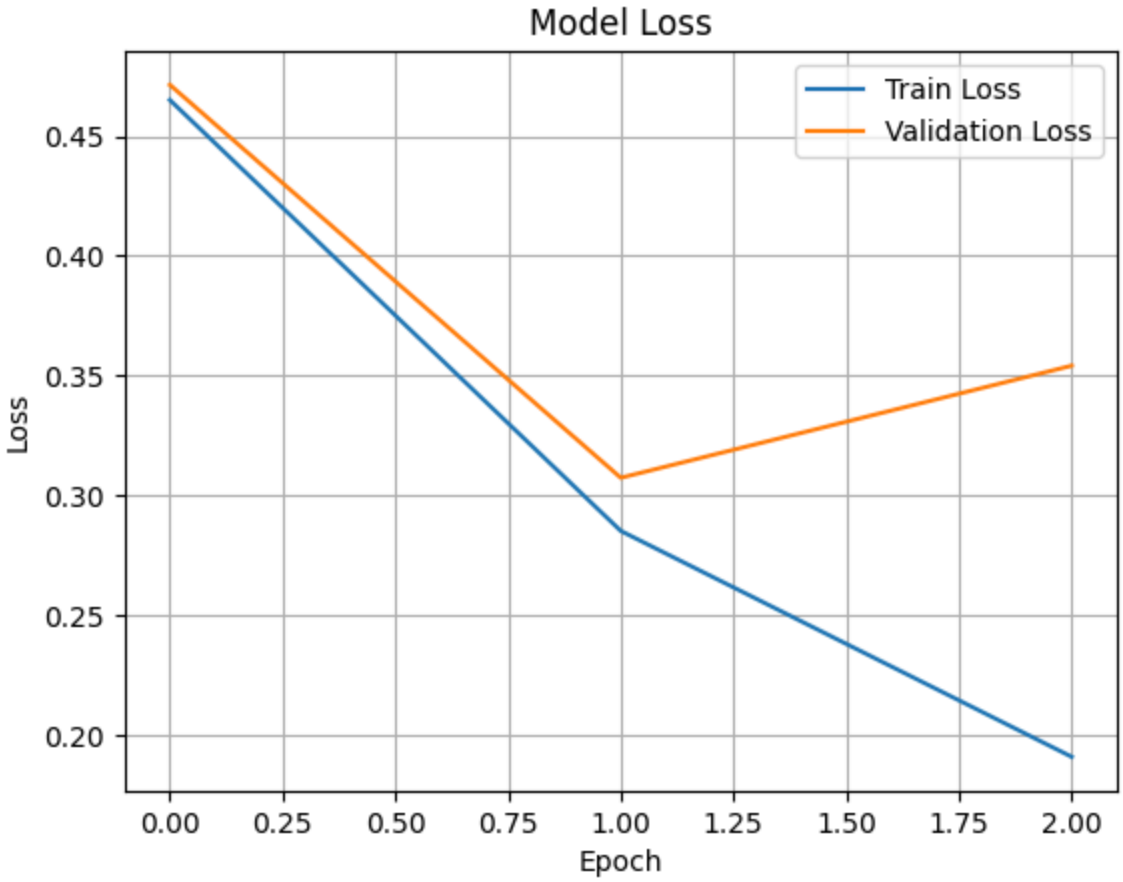
In [11]:
```python
best_model = max(results, key=lambda x: x['test_accuracy'])

print("\n BEST CONFIGURATION:")
print(best_model)
```

```
 BEST CONFIGURATION:
{'embedding_dim': 128, 'lstm_units': 32, 'dropout': 0.5, 'test_accuracy': 0.869
1200017929077}
```

In [12]:
```python
import matplotlib.pyplot as plt
```

```
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])

plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')

plt.legend(['Train Accuracy', 'Validation Accuracy'])
plt.show()



plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')

plt.legend(['Train Loss', 'Validation Loss'])
plt.grid()
plt.show()
```




Model Loss
Train Loss
Validation Loss
Loss
Epoch
0.45
0.40
0.35
0.30
0.25
0.20
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00